

Root Searcher and Implied Volatility Surface

QF607 Numerical Methods

Zhenke Guan

zhenkeguan@smu.edu.sg

Outline

- Black-Scholes model recap - Implied Vol from Price
- Root Finding Method
 - ▶ Bisection
 - ▶ Secant
 - ▶ False Position
 - ▶ Brent
 - ▶ Newton
- Implied Volatility Surface - Interpolation
- FX Market Conventions: Spot/Delta/ATMVol/RR/BF

Black-Scholes Model

- We have been dealing with the Black-Scholes model

$$\frac{dS}{S} = (r - q)dt + \sigma dW \quad (1)$$

- Constant drift $r - q$ and volatility σ
- If the model indeed describes how the market behaves, any derivative product based on the underlying asset S is *almost* risk-free
- Unfortunately the market does not behave so
- Does it make Black-Scholes model useless? No.

- Vanilla options are very liquid in the market
- The market agrees on the prices of them for certain maturities and certain strikes
- Financial institutes, market makers, receive requests for options of any strike and any maturity
- Maintaining directly their prices is not quite convenient since it depends highly on the underlying asset's spot price which is moving every second

↳ also difficult to interpolate as it needs to be efficient and arbitrage-free

- Black-Scholes model comes handy in this situation
- It provides with formula for option prices

$$V_{call}(K, S, \sigma, T, r) = SN(d_+) - e^{-rT} KN(d_-) \quad (2)$$

where

$$d_{\pm} = \frac{\ln \frac{S}{K} + (r - q)T \pm \frac{1}{2}\sigma^2 T}{\sigma\sqrt{T}}$$

only thing not observable and need to be interpreted.

- In the formula, everything are known from the market and trade except the “volatility”.
- If we know the price of the call option from the market, we are able to back out the corresponding volatility, and this is called Black-Scholes **implied volatility**

- Given all the other variables, we can look at the Black-Scholes formula as a function from volatility σ to the call option price V :

$$f_{\underbrace{\{S, r, q, K, T\}}_{\text{observable}}}(\sigma) = V$$

- So the implied volatility is

$$\sigma = f^{-1}(V)$$

- The call option price is a monotonic function to the volatility — time value due to volatility increases when σ increases, when $T > 0$
- However, f^{-1} has no closed form, (2) is an implicit function of σ
- We need to resort to a **root finding algorithm** to back out the price from call option prices

Root Finding

Given a function f , find the x such that

$$f(x) = 0$$

← model price matches market price
← model implied vol - market observable vol = 0.

(3)

- The problem can be one dimensional or N dimensional.
- One dimensional problem is easier and there are robust algorithms
- N dimensional problem is much more difficult and require a lot of insight of the function itself
- We mainly deal with one dimensional problem

Bracketing The Root

- Most root finding algorithms require a bracket that contains the root

- An interval (a, b) brackets the root if $f(a)$ and $f(b)$ have opposite signs, provided that the function $f(x)$ is **continuous**
 - so that solution lies between a and b .
 - if a and b are in opposite direction they make a valid interval.
- We can bracket the root by starting with an initial interval, expanding it outward or inward until $f(a)f(b) \leq 0$

Root Bracketing Algorithm

Algorithm 1 $(a, b) = \text{RootBracketing}(f, a, b, \text{maxIter}, \text{factor})$

```
1: for  $k = 1$  to  $\text{maxIter}$  do  
2:   if  $f(a)f(b) < 0$  then  $\rightarrow$   $a$  and  $b$  different direction (if  $a \times b = -ve$ )  
3:     return  $(a, b)$   $\text{so interval}$   
4:   end if  
5:   if  $\text{abs}(f(a)) < \text{abs}(f(b))$  then  
6:      $a+ = \text{factor} * (a - b)$  // if  $f(a)$  is closer to 0, change  $a$   
7:   else  
8:      $b+ = \text{factor} * (b - a)$  // if  $f(b)$  is closer to 0, change  $b$   
9:   end if  
10: end for
```

- if $\text{factor} > 0$ the algorithm search outward
- if $-1 < \text{factor} < 0$ the algorithm search inward

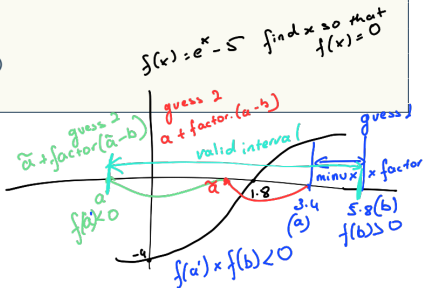
Root Bracketing Implementation

```
1 def rootBracketing(f, a, b, maxIter, factor):
2     for k in range(maxIter):
3         if f(a) * f(b) < 0:
4             return (a, b)
5         if abs(f(a)) < abs(f(b)):
6             a += factor * (a-b) # if f(a) is closer to 0, change a
7         else:
8             b += factor * (b-a) # if f(b) is closer to 0, change b
9     return (a, b)
```

```
1 foo = lambda x : math.exp(x) - 5
2 a = 3.4
3 b = 5.78
4 (a_, b_) = rootBracketing(foo, a, b, 50, 1.6)
5 print(a_, b_)
```

Output:

-0.408000000000000008 5.78



Root Finding Algorithms

Now we are equipped with a root bracketing algorithm. Back to our root finding problem

Root Finding

Given a function f , find the x such that

$$f(x) = 0$$

Commonly used root finding algorithms:

- Bisection method
- Secant method
- False position method (regula falsi)
- Brent's method
- Newton method

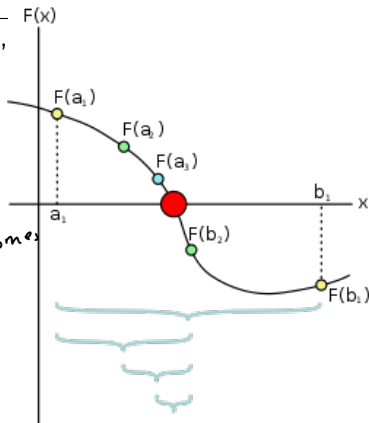
Bisection Method

Start with $[a, b]$ that brackets the root, cut it by half, then see which half contains the root.

Algorithm 2 $\text{root} = \text{rfbisect}(f, a, b, \text{tol})$

Require: $f(a)f(b) < 0$, f is continuous, ,
tolerance tol

```
1: while  $(b - a)/2 > \text{tol}$  do
2:    $c \leftarrow \frac{a+b}{2}$  ← find middle point
3:   if  $f(c) == 0$  then ← check if solution
4:     return  $c$ ;
5:   else if  $f(a)f(c) < 0$  then
6:      $b \leftarrow c$  ← if  $f(a)f(c) < 0$  then  $c$  becomes new  $b$ 
7:   else
8:      $a \leftarrow c$  if not  $c$  becomes new  $a$ .
9:   end if
10: end while
```



Bisection Method — Implementation

Bisection method

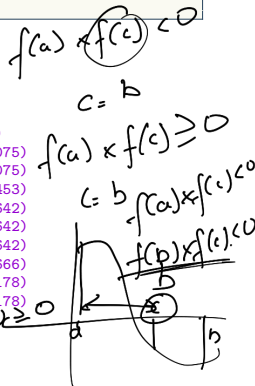
```
1 def bisect(f, a, b, tol):
2     assert(a < b, f(a) * f(b) < 0)
3     c = (a+b) / 2
4     while (b-a)/2 > tol:
5         c = (a+b)/2
6         if abs(f(c)) < tol:
7             return c
8         else:
9             if f(a) * f(c) < 0:
10                 b = c
11             else:
12                 a = c
13     return c
```

Test

```
1 # bs price for 10% vol
2 price = bsPrice(S=100, r=0.02, vol=0.1, T=1.0,
3               strike=90, payoffType=PayoffType.Call)
4 f = lambda vol: (bsPrice(100, 0.02, vol, 1.0, 90,
5               PayoffType.Call) - price)
6 a, b = 0.0001, 0.5
7 iv = bisect(f, a, b, 1e-6)
8 print("implied vol = ", iv)
```

Output:

```
(a, b) = (0.0001, 0.5)
(a, b) = (0.0001, 0.25005)
(a, b) = (0.0001, 0.125075)
(a, b) = (0.0625875, 0.125075)
(a, b) = (0.0938312, 0.125075)
(a, b) = (0.0938312, 0.109453)
(a, b) = (0.0938312, 0.101642)
(a, b) = (0.0977367, 0.101642)
(a, b) = (0.0996895, 0.101642)
(a, b) = (0.0996895, 0.100666)
(a, b) = (0.0996895, 0.100178)
(a, b) = (0.0999335, 0.100178)
```



Bisection Method — Pro's and Con's

- Easy to understand and implement
- It cannot fail in theory — very robust
 - ▶ If the interval happens to contain more than one root, bisection will find one of them.
 - ▶ If the interval contains no roots and merely straddles a singularity, it will converge on the singularity.
- Linear convergence:

$$\epsilon_{n+1} = \text{constant} \times \epsilon_n^m, \quad m = 1 \quad (4)$$

or the log of error changes *linearly*

Secant Method $\leftarrow$ more efficient

Start with x_1 and x_2 , connect $(x_1, f(x_1))$ and $(x_2, f(x_2))$, find the intersection with x axis at x_3 , then continue with x_2 and x_3

Algorithm 3 root = rfsecant(f, x_1, x_2, tol)

Require: f is continuous, tolerance tol

1: **while** $abs(x_2 - x_1) > tol$ **do**

2: $x_3 \leftarrow \frac{x_1 f(x_2) - x_2 f(x_1)}{f(x_2) - f(x_1)}$

3: **if** $f(x_3) == 0$ **then**

4: return x_3 ;

5: **else**

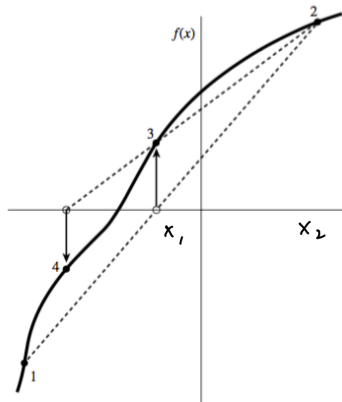
6: $x_1 \leftarrow x_2$

7: $x_2 \leftarrow x_3$

8: **end if**

9: **end while**

move out of the initial bracket.



Secant Method — Implementation

Secant method

```
1 def secant(f, a, b, tol, maxIter):
2     nIter = 0
3     c = (a*f(b)-b*f(a))/(f(b)-f(a))
4     while abs(a - b) > tol and
5         nIter <= maxIter:
6         c = (a*f(b)-b*f(a))/(f(b)-f
7             (a))
8         if abs(f(c)) < tol:
9             return c
10        else:
11            a = b
12            b = c
13            nIter = nIter+1
14    return c
```

Test

```
1 price = bsPrice(S=100, r=0.02, vol=0.1, T=1.0,
2             strike=90, payoffType=PayoffType.Call)
3 f = lambda vol: (bsPrice(100, 0.02, vol, 1.0,
4             90, PayoffType.Call) - price)
5 a, b = 0.0001, 0.5
iv = secant(f, a, b, 1e-6, 100)
print("implied vol = ", iv)
```

Output:

```
(a, b) = (0.0001, 0.5)
(a, b) = (0.5, 0.0178693)
(a, b) = (0.0178693, 0.0350069)
(a, b) = (0.0350069, 58.2528)
(a, b) = (58.2528, 0.345852)
(a, b) = (0.345852, -4.97794)
(a, b) = (-4.97794, -0.0229199)
(a, b) = (-0.0229199, 0.674837)
(a, b) = (0.674837, 0.251415)
(a, b) = (0.251415, 0.130876)
(a, b) = (0.130876, 0.109296)
(a, b) = (0.109296, 0.101454)
(a, b) = (0.101454, 0.100093)
(a, b) = (0.100093, 0.100001)
```


Secant Method — Pro's and Con's

- The secant method is a variant of Newton's method that avoids the use of the derivative of $f(x)$, which can be very helpful when dealing with the derivative is not easy.
- Open bracket method — does not start with a bracket that contains the root
- Order of convergence: $m = 1.62$
- Works very well when the function is locally close to linear
- Convergence is not always guaranteed, need to add `maxIter` in the implementation

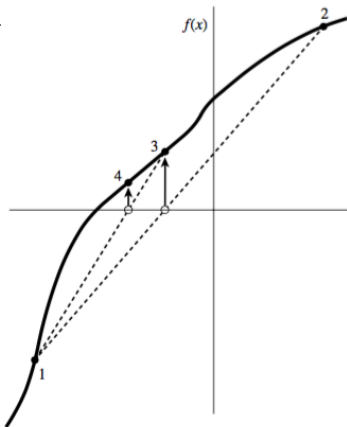
False Position Method (Regula Falsi)

Start with $[a, b]$ that brackets the root, connect $(a, f(a))$ and $(b, f(b))$, find the intersection with x axis at c , then continue with a or b , depending on who brackets the root with c

Algorithm 4 $\text{root} = \text{rffalsi}(f, a, b, \text{tol})$

Require: $f(a)f(b) < 0$, f is continuous

```
1: while  $b - a > \text{tol}$  do  
2:    $c \leftarrow \frac{af(b) - bf(a)}{f(b) - f(a)}$   
3:   if  $f(c) == 0$  then  
4:     return  $c$ ;  
5:   else if  $f(a)f(c) < 0$  then  
6:      $b \leftarrow c$   
7:   else  
8:      $a \leftarrow c$   
9:   end if  
10: end while
```



False Position Method — Implementation

False position method

```
1 def falsi(f, a, b, tol):
2     assert (a<b and f(a)*f(b)<0)
3     c = (a*f(b)-b*f(a))/(f(b)-f(a))
4     while abs(a - b) > tol:
5         c = (a*f(b)-b*f(a))/(f(b)-f(a))
6         if abs(f(c)) < tol:
7             return c;
8         else:
9             if f(a)*f(c)<0:
10                 b = c
11             else:
12                 a = c
13     return c
```

Test

```
falsi(f, a, b, 1e-6)
```

Output:

```
(a, b) = (0.0001, 0.5)
(a, b) = (0.0178693, 0.5)
(a, b) = (0.0350069, 0.5)
(a, b) = (0.0515307, 0.5)
(a, b) = (0.0670825, 0.5)
(a, b) = (0.0800797, 0.5)
(a, b) = (0.0891336, 0.5)
(a, b) = (0.0944855, 0.5)
(a, b) = (0.0973167, 0.5)
(a, b) = (0.0987226, 0.5)
(a, b) = (0.0993985, 0.5)
(a, b) = (0.0997182, 0.5)
(a, b) = (0.0998683, 0.5)
(a, b) = (0.0999385, 0.5)
(a, b) = (0.0999713, 0.5)
(a, b) = (0.0999866, 0.5)
(a, b) = (0.0999938, 0.5)
(a, b) = (0.0999971, 0.5)
(a, b) = (0.0999986, 0.5)
(a, b) = (0.0999994, 0.5)
(a, b) = (0.0999997, 0.5)
(a, b) = (0.0999999, 0.5)
(a, b) = (0.0999999, 0.5)
```

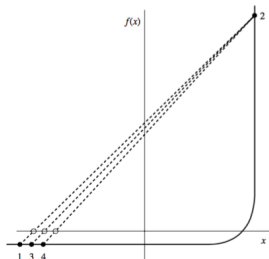
False Position Method — Pro's and Con's

- More robust than secant method — won't shoot outside and root is always bracketed
- If a solution exists it guarantees to find it
- Converges faster than bisection method (usually but not always)
- Often super-linear convergence, but exact order is hard to say

Root Finding Algorithms — Considerations

- Numerical methods are sensitive to
 - ▶ Inputs
 - ▶ Assumptions
 - ▶ The problem itself
 - ▶ Tolerance
 - ▶ Round-off errors, truncation errors, etc.
- Before applying an algorithm, we need to analyse our problem carefully

Example where both the secant and false-position methods will take many iterations to arrive at the true root. This function would be difficult for many other root-finding methods.



Brent's Method

- We have bisection method that is robust but converges linearly
- Secant, false position method that converge faster but might hit bad cases
- Natural improvement is to mix them, at each iteration, when Secant or its like work well, use them, otherwise fall back to bisection
- Brent is the most popular one following this logic
- Robust and efficient, normally **the** root finder in practice

Brent's Method

- Brent method is a mixture of
 - ▶ Bisection method
 - ▶ Secant method
 - ▶ Inverse quadratic interpolation: solve the quadratic function $q(y) = x$ that passes the last three iteration points, then pick the $x = q(0)$
- The logic is to use inverse quadratic interpolation when some conditions are satisfied, and bisection method otherwise
- It requires the roots to be bracketed by the input $[a, b]$
- Detailed algorithm can be found on Wikipedia
https://en.wikipedia.org/wiki/Brent%27s_method

Brent's Method — Implementation

- We use the implementation from `scipy`:

```
1 from scipy import optimize
2
3 price = bsPrice(S=100, r=0.02, vol=0.1, T=1.0, strike=90, payoffType=PayoffType.Call)
4 f = lambda vol: (bsPrice(100, 0.02, vol, 1.0, 90, PayoffType.Call) - price)
5 a, b = 0.0001, 0.5
6 iv = optimize.brentq(f, a, b)
7 print("implied vol = ", iv)
```

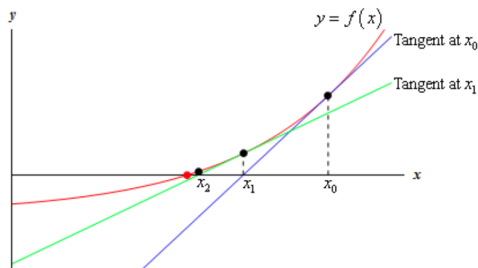
- Convergence is the best among the tests:

```
(a,b) = ( 0.0001 , 0.5 )
(a,b) = ( 0.0001 , 0.01786929710806077 )
(a,b) = ( 0.01786929710806077 , 0.25893464855403037 )
(a,b) = ( 0.01786929710806077 , 0.041003245337575184 )
(a,b) = ( 0.041003245337575184 , 0.14996894694580276 )
(a,b) = ( 0.041003245337575184 , 0.07331686300283635 )
(a,b) = ( 0.07331686300283635 , 0.11164290497431956 )
(a,b) = ( 0.11164290497431956 , 0.09698355896250532 )
(a,b) = ( 0.09698355896250532 , 0.09975150660147088 )
(a,b) = ( 0.09975150660147088 , 0.10000123502161626 )
(a,b) = ( 0.10000123502161626 , 0.09999999758482866 )
(a,b) = ( 0.09999999758482866 , 0.10000049758482871 )
```


Newton's Method

- Newton's method (or Newton-Raphson method) starts with one point instead of two
- At every iteration it updates it with the interception of the tangent line and the x axis:

$$x_{n+1} = x_n - \frac{f(x)}{f'(x)} \quad (5)$$



- Coming from the Taylor series expansion, eliminating higher order terms:

$$f(x + \delta) \approx f(x) + f'(x)\delta + \frac{f''(x)}{2}\delta^2 + \dots$$

Newton's Method

Ex am
ques →

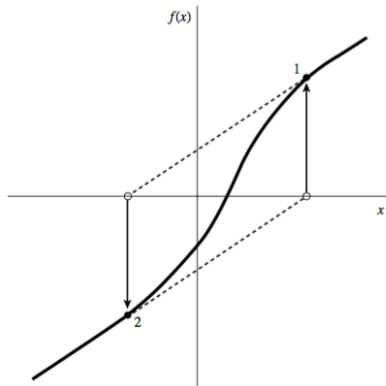
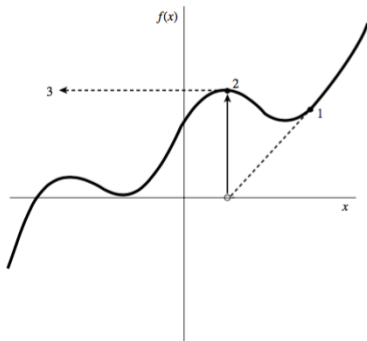
Potential question
 $f(x) = e^x - 5$

- ① Cal 4 steps at bisection (0,3)
- ② Cal 3 steps secant

- Similar to secant method, where $f'(x)$ is replaced by finite difference

$$\frac{f(x_n) - f(x_{n-1})}{x_n - x_{n-1}}$$

- Possible to go wrong:



- Solution? Mix with bisection method

Newton's Method in Multiple Dimension

- Newton's method can be generalized to multi-dimension root search problem:

$$\mathbf{x}_{n+1} = \mathbf{x}_n - \mathbf{J}^{-1}(\mathbf{x})\mathbf{f}(\mathbf{x}) \quad (6)$$

where $\mathbf{J}(\mathbf{x})$ is the Jacobian matrix at $\mathbf{x}$, whose element J_{ij} is

$$J_{ij} = \frac{\partial f_i}{\partial x_j}$$

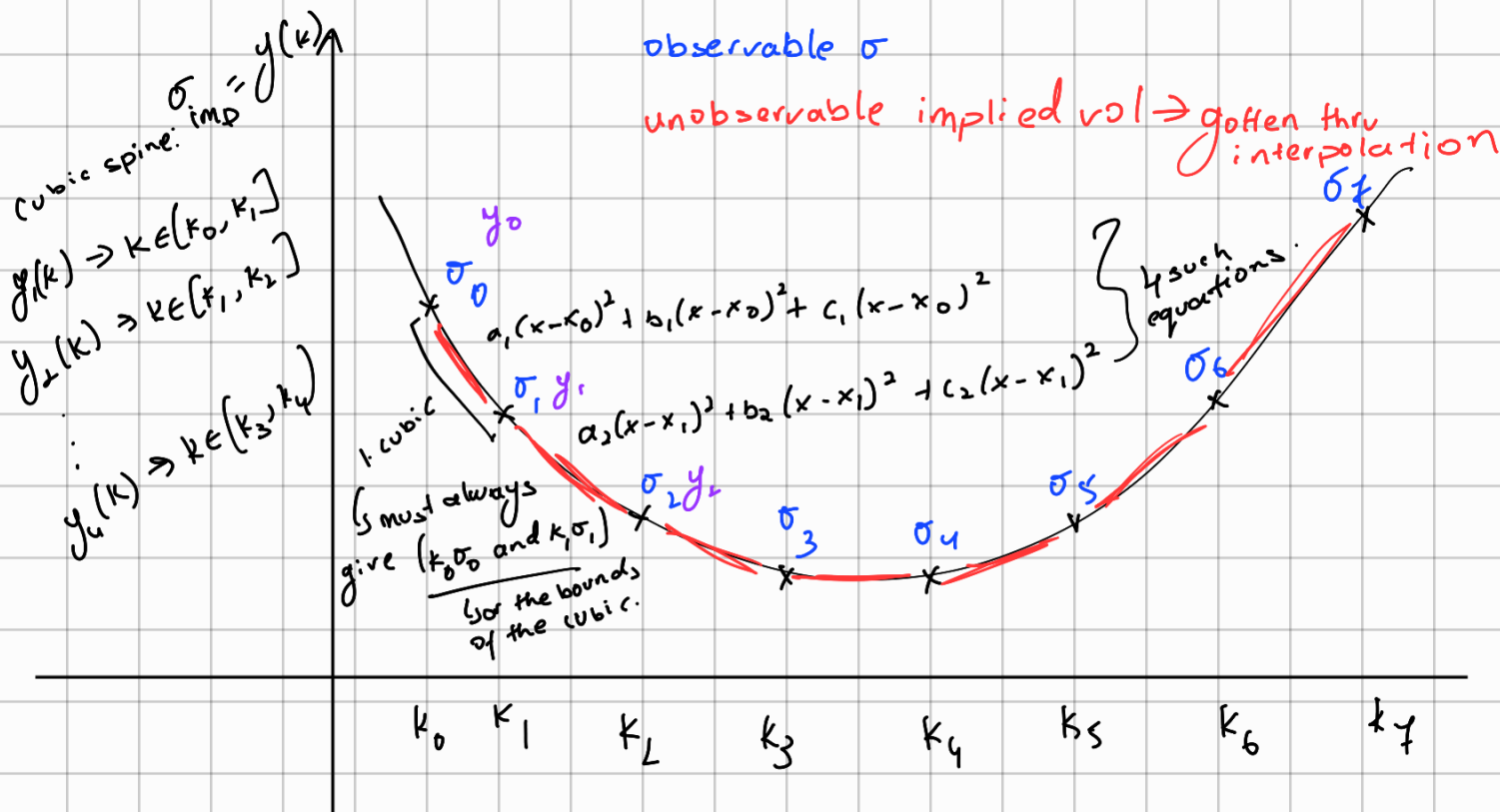
- We do not encounter many N dimensional root finding problems in our field
- Instead, we very often have problems on N dimension minimization / maximization — root finding on gradient vectors, but gradients are not arbitrary functions, they are highly restrictive (obey integrability condition) and well formed

FX Option Market Making

- Vol Smile Interpolation
- Conventions

Back To Market Making

- Now we are able to convert an option price to its implied volatility
- It's common practice to quote options using implied volatilities
 - ▶ Less dependent on change of spot price
 - ▶ The implied volatilities do not change as frequently as the spot price changes
- Now assume we have discrete set of liquid maturities, e.g., 1W, 2W, 1M, 2M, 6M, 1Y, etc.
- And for each maturity (or pillar), we have a sample set of strike to option price tuples, say 5 points, and we used our root finder to convert them to strike to implied volatility tuples: (k_i, σ_i)
- We need to construct an implied volatility surface so that we can quote / price vanilla option at any strike and maturity — we need to **interpolate** and **extrapolate** from our sample points



He may ask us this

$4 \times 2 = 8$ unknowns

$$\begin{aligned}
 & \int_{x \in (x_0, x_1)} f(x) = a(x-x_0)^3 + b_1(x-x_0)^2 + c_1(x-x_0) + d_1 \\
 & \int_{x \in [x_0, x_2]} f(x) = \int_2(x) = a_2(x-x_1)^3 + b_2(x-x_1)^2 + c_2(x-x_1) + d_2
 \end{aligned}$$

x is the one I am trying to find

$$f(x_0) = y_0$$

$$f_1(x_1) = y_1$$

$$f_2(x_1) = y_1$$

$$f_2(x_2) = y_2$$

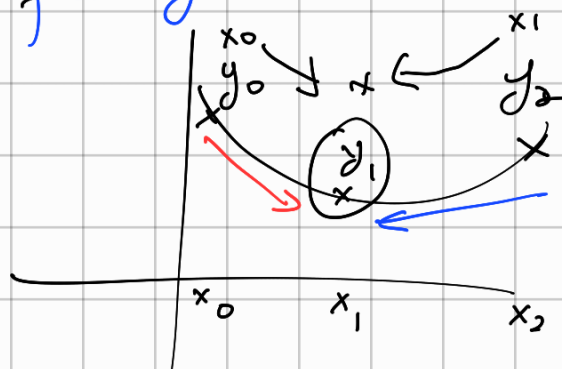
$\rightarrow$ approaching from left +

$\rightarrow$ approaching from right



$$f_1'(x_1) = f_2'(x_1)$$

$$f_1''(x_1) = f_2''(x_1)$$



$\frac{1^{st}}{f_1'(x_1)/f_1''(x_1)}$ deriv approaching from left + $= \frac{1^{st}}{f_2'(x_1)/f_2''(x_1)}$ deriv approaching from right

$f_1''(x_0) = 0$
 $f_2''(x_2) = 0$ $\leftarrow$ why this?

Volatility Smile Interpolation

- The function $k \rightarrow \sigma$ for one maturity is often called volatility smile, due to its convex shape
- One common way to interpolate volatility smile is to use cubic spline interpolation
- Reason to choose cubic spline:
 - ▶ It is C^2 — smooth in first derivative and continuous in second derivative, both within the interval and at the boundaries
 - ▶ Its coefficients are fast to compute

Given sample points $(x_i, y_i), i \in [0, 1, \dots, N]$. Let $y(x)$ be given by a different cubic polynomial in each interval $[x_0, x_1], [x_1, x_2], \dots, [x_{N-1}, x_N]$. Let $y(x)$ be given by $y_j(x)$ if $x \in [x_j, x_{j+1}]$.

Cubic Spline Interpolation

make sure.
→ same number of unknowns
and same number of equations

Each cubic polynomial is defined by 4 coefficients so we have a total $4N$ parameters. They are determined by the following conditions:

- $y(x)$ must interpolate the data points so in each subinterval $j = 0, \dots, N - 1$, we must have $y_j(x_j) = y_j$ and $y_j(x_{j+1}) = y_{j+1}$.
- $y'(x)$ must be continuous at each of the internal knots. Therefore for $j = 1, 2, \dots, N - 1$ we must have $y'_{j-1}(x_j) = y'_j(x_j)$.
first derivative of the cubic function must be same
- $y''(x)$ must be continuous at each of the internal knots. Therefore for $j = 1, 2, \dots, N - 1$ we must have $y''_{j-1}(x_j) = y''_j(x_j)$.
even 2nd derivative has to be same.
- A choice of one of the 2 conditions at the 2 end points
if not equal at the joining points then jagged → some derivative then smoother

Condition 1 gives $2N$ relations. Conditions 2, 3 each gives $N - 1$ relations. Together with the 2 relations from condition 4, we have $4N$ conditions. Thus we have just the right number of relations to determine all the parameters uniquely.

Cubic Spline Interpolation

- Start with linear interpolation, for $x \in [x_j, x_{j+1}]$

$$y = Ay_j + By_{j+1}, \quad A = \frac{x_{j+1} - x}{x_{j+1} - x_j}, \quad B = 1 - A \quad (7)$$

- Impose second derivatives y''

$$y = Ay_j + By_{j+1} + Cy_j'' + Dy_{j+1}'' \quad (8)$$

$$C = \frac{1}{6}(A^3 - A)(x_{j+1} - x_j)^2, \quad D = \frac{1}{6}(B^3 - B)(x_{j+1} - x_j)^2 \quad (9)$$

- The choice of C and D make the second derivative of y with respect to x the linearly interpolated value of y_j'' and y_{j+1}''

$$\frac{d^2y}{dx^2} = Ay_j'' + By_{j+1}'' \quad (10)$$

So second derivative is continuous. Notice that the dependence on the independent variable x in equations above is entirely through the linear x -dependence of A and B , and (through A and B) the cubic x -dependence of C and D .

Cubic Spline Interpolation

- Our second purpose: first derivative is smooth. It is already the case inside each segment, so we only need the left and right first derivative at each sample point to be equal.
- First derivative in segment $[x_j, x_{j+1}]$ is

$$\frac{dy}{dx} = \frac{y_{j+1} - y_j}{x_{j+1} - x_j} - \frac{3A^2 - 1}{6}(x_{j+1} - x_j)y_j'' + \frac{3B^2 - 1}{6}(x_{j+1} - x_j)y_{j+1}'' \quad (11)$$

- Equalizing $\frac{dy}{dx}$ calculated from left and right segments at the junction points, we get $N - 2$ equations
- For N unknowns y_j'' , we are left with two degrees of freedom
 - ▶ Setting y_0'' and y_{N-1}'' to 0 — natural cubic spline
 - ▶ Calculate y_0'' and y_{N-1}'' such that y_0' and y_{N-1}' equal to specified value

Cubic Spline In Our Application

- We may set y'_0 and y'_{N-1} to 0 as boundary conditions because we want our volatility to be flattened beyond the boundary (clamped spline)
- We can choose to interpolate σ^2 (variance) or σ (volatility), no particular advantage of either one
- Note that cubic spline can generate negative values — does not happen in ordinary markets where volatility is not too low
- Also note that there is no guarantee that the interpolated implied volatilities may contain arbitrage — problem for pegged currencies

Time Interpolation

- Now for each maturity we have a volatility smile
- When we need to evaluate options expiring not exactly on a pillar date, we need to interpolate our implied volatility in time
- Time interpolation is easy, it can be linear on variance along the strike line (moneyness line to be precise)
 - ▶ For options with the same strike, the longer the maturity the higher the option value — calendar arbitrage otherwise
 - ▶ Interpolating on variance along the strike line makes sure that the variance will be increasing, as long as it is for the same K the next pillar has a higher variance than the previous pillar
- Mathematically,

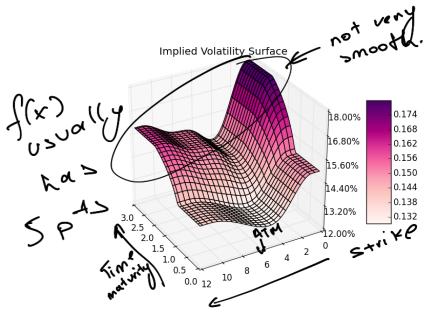
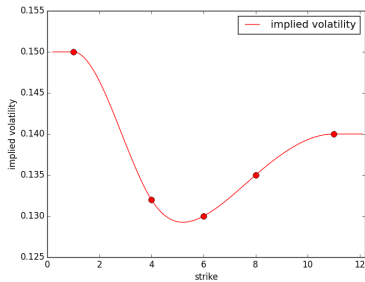
$$v_t(K) = \frac{t_{i+1} - t}{t_{i+1} - t_i} v_{t_i}(K) + \frac{t - t_i}{t_{i+1} - t_i} v_{t_{i+1}}(K), \quad v_t(K) = \sigma_t^2(K)t \quad (12)$$

Implied Volatility Surface Implementation

- Implied volatility surface and smile builder: it takes as input the strike to implied volatility pairs for each pillar, and provide with function that returns implied volatility for a given strike and maturity:

```
1 class Smile:
2     def __init__(self, strikes, vols):
3         self.strikes = strikes
4         self.vols = vols
5         self.cs = CubicSpline(strikes, vols, bc_type=((1, 0.0), (1, 0.0)), extrapolate=True)
6     def Vol(self, k):
7         if k < self.strikes[0]: # scipy cubicspline bc_type confusing, extrapolate by ourselves
8             return self.vols[0]
9         if k > self.strikes[-1]:
10            return self.vols[-1]
11        else:
12            return self.cs(k)
13
14 class ImpliedVol:
15     def __init__(self, ts, smiles):
16         self.ts = ts
17         self.smiles = smiles
18     # linear interpolation in variance, along the strike line
19     def Vol(self, t, k):
20         ... # look up the interval t is in, and interpolate
```

Interpolated Implied Volatility Smile and Surface



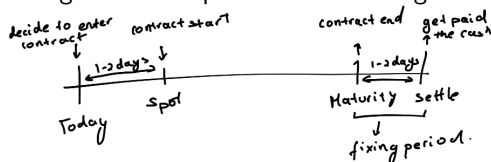
- Interpolated smile (left) $\hookrightarrow$ maturity and volatility surface (right) $\hookrightarrow$ many maturity
- Cubicspline for smile interpolation, linear on variance for time interpolation

We Are Not Done Yet

- If the market quotes us sample set of **strike** to option price pairs, we can now construct our implied volatility surface, with that we can in-house vanilla option with any maturity
- Markets of some asset classes do provide you with those, e.g., some commodities and equity, interest rates market (there would be other complications though)
- Some market does not quote you those. For example, FX market quotes
 - ▶ at-the-money volatility
 - ▶ volatility differentials — risk reversals and butterflies at 25 and 10 delta points (such that the strike dimension is more homogeneous)
- We need an extra step to convert our market marks to the inputs of our implied volatility constructor. **Root finders** come very handy here.

FX Market Conventions — Spot

- Spot: the exchange rate between two currencies
 - ▶ In practice the FX spot rate refers to the rate for exchanging two currencies in one or two days later, this is called the spot-lag
 - ▶ Spot-lag makes the spot rate effectively a forward rate, and makes things a little bit more complicated in implementation
 - ▶ We ignore the spot-lag and treat spot as cash exchange rate in this course



FX Market Conventions — ATMVOL

- ATMVOL: the implied volatility for at-the-money options
- Not necessarily mean $K_{ATM} = S$ (usually not the case in FX):
 - ▶ **ATM forward**, ATM strike is the forward:

$$K_{ATM} = F = Se^{\int_0^t (r_d(s) - r_f(s)) ds} \quad (13)$$

“ATM call and put options have the same price.” Here the ATM option refers to ATM forward.

- ▶ **ATM Delta-neutral**: $K_{ATM} = K_{DNS}$ the strike at which the straddle corresponds to a pure long Vega position with no net Delta:

$$\Delta(\text{Call}, K_{DNS}, T, \sigma_{ATM}) + \Delta(\text{Put}, K_{DNS}, T, \sigma_{ATM}) = 0 \quad (14)$$

Delta neutral straddle (DNS)
 ↳ find K such that this equation satisfies so

↳ these 2 parameters same for call and put
 ↳ diff sign from call

$$N\left(\frac{\ln \frac{F}{K_{DNS}} + \frac{1}{2} \sigma_{ATM}^2 T}{\sigma_{ATM} \sqrt{T}}\right) - N\left(\frac{-\ln \frac{F}{K_{DNS}} - \frac{1}{2} \sigma_{ATM}^2 T}{\sigma_{ATM} \sqrt{T}}\right) = 0,$$

$$K_{DNS} = Fe^{\frac{1}{2} \sigma_{ATM}^2 T}. \quad (15)$$

FX Market Conventions — Delta

- Delta is the sensitivity of option price to the underlying
- Recall that Black-Scholes call formula for an FX option

$$Call_{BS} = Se^{-r_f T} N(d_+) - Ke^{-r_d T} N(d_-) \quad (16)$$

$$d_{\pm} = \frac{\ln(F/K) \pm \frac{1}{2}\sigma^2 T}{\sigma\sqrt{T}} \quad (17)$$

where r_f is the risk free rate of the LHS currency (foreign ccy), r_d is the risk-free interest rate of the RHS currency (domestic ccy)

- Call delta:

$$\Delta_{call}(\sigma, K, T, r_d, r_f) = \frac{\partial Call_{BS}}{\partial S} = e^{-r_f T} N(d_+) \quad (18)$$

FX Market Conventions — Delta

- Similarly the Black-Scholes put formula

$$Put_{BS} = Ke^{-r_d T} N(-d_-) - Se^{-r_f T} N(-d_+) \quad (19)$$

- And put delta

$$\Delta_{put}(\sigma, K, T, r_d, r_f) = \frac{\partial Put_{BS}}{\partial S} = -e^{-r_f T} N(-d_+) \quad (20)$$

FX Market Conventions — Spot Delta and Forward Delta

- Similar to ATM convention, Delta has its conventions
- What we have seen are the first derivative to the spot — **spot Delta**
- It can be also the first derivative with respect to the forward — **forward Delta**
- Forward Delta is the first derivative of the **undiscounted price** with respect to the forward

$$\text{Black mode!} \left\{ \begin{aligned} \Delta_{call} &= \frac{\partial [FN(d_+) - KN(d_-)]}{\partial F} = N(d_+) & (21) \\ \Delta_{put} &= \frac{\partial [KN(-d_-) - FN(-d_+)]}{\partial F} = -N(-d_+) & (22) \end{aligned} \right.$$

FX Market Convention — Our Use Case

We will use **ATM forward** convention and **forward Delta** for illustration

- The forward is more the center of the strike, rather than spot, especially for long dated options

$$K_{ATM}(t) = F(t) = Se^{(r_d - r_f)t} \quad (23)$$

*only need
if statement*

- Forward delta ranges from 0 to 1 — the cumulative normal function

$$\Delta_{call} = N(d_+), \quad \Delta_{put} = -N(-d_+) \quad (24)$$

- Simple in form, without the tedious discount factor
- Keep in mind that an FX market in practice handles all the hassles regarding the conventions

- Also note that we have omitted one convention — the **premium convention**. All our premium are quoted in the the RHS (domestic) currency. If the option premium is quoted in the LHS currency, the delta will change since the unit of delta is the LHS currency

*ignore
this
convention*

Why Deltas?

- It is the first component of your risk to be hedged
- It is used as a homogeneous measure of moneyness across option maturities. A 1Y call and a 10Y call, both struck at 100 does not have the same delta risk — forward curve is not flat
- The market players often quote like 1Y 25d call, 20%, meaning the call option with the strike K such that $\Delta_{call}(\sigma = 20\%) = 0.25$. The exact strike K needs to be found through a root finder.
- Most liquid FX option instruments have strikes at call/put, 25d/10d

FX Market Instruments

- Ideally if the market quotes delta to implied volatility pairs (d_i, σ_i) we can just convert them to strike and implied volatility pairs (K_i, σ_i) and we will have the inputs for our implied volatility surface interpolater.
- Unfortunately except ATMVOL, most likely you will get quotes for volatility spreads, in the form of **market strangle** or **smile strangle**, and **risk reversal**

Market strangle is a volatility spread $\sigma_{\Delta_{ms}}$ used to represent a strangle payoff: a call + a put option (with the same expiration date but different strikes).

- The strike of the call option is calculated as the strike K_c such that

$$\Delta_{call}(K_c, \sigma_{ATM} + \sigma_{\Delta_{ms}}) = \Delta \quad (25)$$

↙ market strangle

- The strike of the put option is calculated as the strike K_p such that

$$\Delta_{put}(K_p, \sigma_{ATM} + \sigma_{\Delta_{ms}}) = -\Delta \quad (26)$$

- The price of the strangle instrument is the sum of the two options using Black-Scholes formula using K_c , K_p , and volatility $\sigma_{ATM} + \sigma_{\Delta_{ms}}$
- Given σ_{ATM} and $\sigma_{\Delta_{ms}}$, the strangle instrument and its price is determined. It forms a constrain of the smile we build.
- Market strangle describes the convexity of the volatility smile
- Typical strangle instruments: 10 or 25 delta, i.e., $\Delta = 0.25$ or 0.10

Smile Strangle and Risk Reversal

Smile strangle BF_{Δ} is the pure volatility spread:

$$BF_{\Delta} = \frac{1}{2}(\sigma_{\Delta_c} + \sigma_{\Delta_p}) - \sigma_{ATM} \quad (27)$$

For example 25 delta smile strangle is the average of 25 call delta's implied volatility and 25 put delta's implied volatility minus σ_{ATM} . It is commonly called the **butterfly**. It measures the convexity of the smile.

Risk reversal RR_{Δ} is the pure volatility spread:

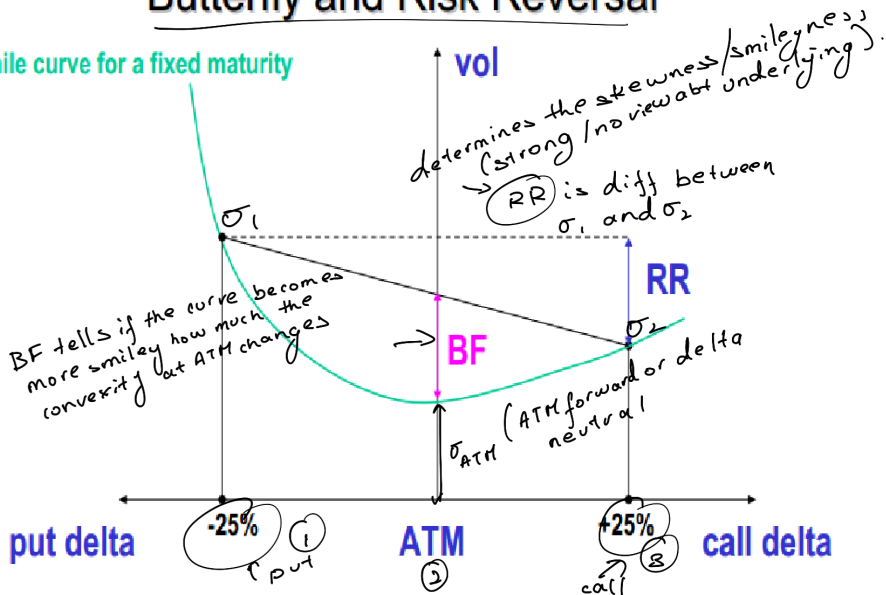
$$RR_{\Delta} = \sigma_{\Delta_c} - \sigma_{\Delta_p} \quad (28)$$

It measures the skewness of the smile.

Let's see it in Graph

Butterfly and Risk Reversal

Smile curve for a fixed maturity



The FX Volatility Marking Problems

- Given σ_{ATM} , if we know the smile strangle and risk reversals for 25 and 10 delta points, we can easily calculate the implied volatilities for the four delta points:

$$\begin{cases} BF_{\Delta} &= \frac{1}{2}(\sigma_{\Delta_c} + \sigma_{\Delta_p}) - \sigma_{ATM} \\ RR_{\Delta} &= \sigma_{\Delta_c} - \sigma_{\Delta_p} \end{cases} \quad (29)$$

— two equations and two unknowns for each delta point

- Converting ^{every single pair} (Δ_i, σ_i) pairs to (K_i, σ_i) pairs, and together with the ATM pairs, we can form the inputs to our implied volatility surface
- Risk reversal is available in the raw market
- Smile strangle is not available. Instead we have only market strangle in the raw market.
- The practical problem is to build implied volatility surface given σ_{ATM} , market strangles, and risk reversals

How About Diffusion Models?

- We are able to price plain vanilla option and match the quotes in the market
- What else can we price? Maybe some digital option with static replication — note that the Black-Scholes digital option formula does not work now, why? *→ wrong assumptions*
- We are model-less now — our only model so far Black-Scholes does not work. We do not have a diffusion model that can reproduce all the vanilla prices, so not possible to price any path dependent exotic products
- This is in fact the basic requirement of any advanced models: they need to be able to calibrate to the implied volatility surface — reproducing the vanilla prices
- The **local volatility model** on the way — the straight forward yet effective extension to Black-Scholes model

References



William H. Press, Saul A. Teukolsky, William T. Vetterling , Brian P. Flannery *Numerical Recipes*, 3rd Edition: The Art of Scientific Computing Hardcover – 6 September 2007



Iain J. Clark *Foreign Exchange Option Pricing: A Practitioner's Guide*, 2010